to understand where a specific method comes from and how it gets into a class such as Base. The rest of this short chapter will look at how ActiveRecord::Base's functionality is assembled.

Let's start by taking a step back to the first line in our example: require 'active_record'.

The Autoloading Mechanism

Here's the code in active_record.rb, the only Active Record file that you're likely to require:

gems/activerecord-4.1.0/lib/active_record.rb

```
require 'active_support'
require 'active_model'
# ...

module ActiveRecord
  extend ActiveSupport::Autoload

  autoload :Base
  autoload :NoTouching
  autoload :Persistence
  autoload :QueryCache
  autoload :Querying
  autoload :Validations
  # ...
```

Active Record relies heavily on two other libraries that it loads straight away: Active Support and Active Model. We'll get to Active Model soon, but one piece of Active Support is already used in this code: the ActiveSupport::Autoload module, which defines an autoload method. This method uses a naming convention to automatically find and require the source code of a module (or class) the first time you use the module's name. Active Record extends ActiveSupport::Autoload, so autoload becomes a class method on the ActiveRecord module itself. (If you're confused by this mechanism, look back at the *Class Extension (130)* spell.)

Active Record then uses autoload as a *Class Macro (117)* to register dozens of modules, a few of which you can see in the code above. As a result, Active Record acts like a smart *Namespace (23)* that automatically loads all the bits and pieces that make up the library. For example, when you use ActiveRecord::Base for the first time, autoload automatically requires the file active_record/base.rb, which in turn defines the class. Let's take a look at this definition.